Software Test Plan (STP)

Real-Time Chat Application (Socket Programming)

Field	Details
Project	Real-Time Chat Application (Socket Programming)
Version	1.0
Team No.	5
Authors	Amoghaa R, Vaibhav R, P S Harshavardhan, Pramath Ganesh Hegde
Date	16-09-2026
Status	Final

Team

Name	SRN	Role / Focus Area
Amoghaa R	PES2UG24CS625	Testing & Documentation
Vaibhav R	PES2UG24CS664	Server / Socket Backend
P S Harshavardhan	PES2UG25CS817	Authentication & Database
Pramath Ganesh Hegde	PES2UG24CS648	Frontend / UI

Revision History

Version	Date	Author	Change Summary	Approval
1.0	16-09-2026	Team 5	Initial STP for Real-Time Chat Application based on SRS v1.0, including defect severity classification, unit-test framework and browser/device matrix, expanded security validation for input sanitization, and a schedule-buffer risk mitigation.	

1. Introduction

Purpose: This document defines the Software Test Plan (STP) for the Real-Time Chat Application v1.0. It explains how the application will be tested and covers the testing scope, strategy, resources, schedule, and responsibilities. The plan is based on the Software Requirements Specification (SRS) v1.0.

Scope: Testing covers user registration and authentication, one-to-one and group real-time messaging, presence and notifications, file/media sharing, message history and search, and administration/moderation. Voice/video calling, end-to-end encryption key management beyond WSS/TLS, and third-party push-notification gateway internals are outside the testing scope.

References: Real-Time Chat Application SRS v1.0 (Team 5) and the Requirements Traceability Matrix (RTM) provided in the SRS.

Definitions: WS (WebSocket), WSS (WebSocket Secure), REST (Representational State Transfer), JWT (JSON Web Token), UI (User Interface), API (Application Programming Interface), TLS (Transport Layer Security), XSS

(Cross-Site Scripting), DoS (Denial of Service), RTM (Requirements Traceability Matrix), FR (Functional Requirement), NFR (Non-Functional Requirement).

2. Test Items

- Authentication & User Management module
- One-to-One Messaging module
- Group Chat module
- Presence & Notifications module
- File & Media Sharing module
- Message History & Search module
- Administration & Moderation module
- WebSocket server and connection handling
- Database persistence
- Responsive web user interface

3. Features to be Tested

The following features are mapped to the requirements in the SRS:

Req ID	Feature	Priority
CHAT-F-001	User registration with unique username/email	High
CHAT-F-002	Authenticate credentials and open WebSocket session	High
CHAT-F-003	Logout / invalidate session	Medium
CHAT-F-010	Real-time one-to-one text message delivery	High
CHAT-F-011	Persist message for offline recipient delivery	High
CHAT-F-012	Delivery and read receipts	Medium
CHAT-F-013	Typing indicator	Low
CHAT-F-020	Create group chat with initial members	High
CHAT-F-021	Add / remove group members	Medium
CHAT-F-022	Broadcast group message to all members	High
CHAT-F-023	Leave a group	Low
CHAT-F-030	Online/offline status and last-seen	Medium
CHAT-F-031	Missed-message notification on reconnect	Medium
CHAT-F-032	Unread-message count per conversation	Low
CHAT-F-040	Upload and share file/image	Medium
CHAT-F-041	Validate attachment size and file type	High
CHAT-F-050	Persist and retrieve chat history	High
CHAT-F-051	Delete own message within time window	Low
CHAT-F-060	Suspend / ban a reported user	Medium
CHAT-F-061	Broadcast system-wide announcement	Low

Req ID	Feature	Priority
CHAT-NF-001	Message delivery latency ≤200 ms (p95, 100 concurrent)	High
CHAT-NF-002	Support at least 100 concurrent WebSocket connections	Medium
CHAT-NF-003	99.5% availability and auto-reconnect ≤5 s	High
CHAT-NF-004	Responsive UI; first message within 3 clicks/taps	Medium
CHAT-NF-005	Modular codebase; ≥80% server-side unit test coverage	Medium
CHAT-SR-001	TLS/WSS for all client-server traffic	High
CHAT-SR-002	Strong password hashing; no plaintext storage	High
CHAT-SR-003	Valid authentication token required for WebSocket handshake	High
CHAT-SR-004	Server-side conversation/group membership check	High
CHAT-SR-005	Rate limiting on message-send events	Medium

4. Features Not to be Tested

- Voice/video calling (outside the SRS scope).
- End-to-end encryption key management beyond transport-layer WSS/TLS security.
- Third-party push-notification gateway internals.
- Dedicated CDN behavior for attachments, because the SRS specifies local/server storage.

5. Test Approach / Strategy

Levels:

- Unit testing – test individual modules such as authentication, messaging, groups, presence, uploads, history, and administration.
- Integration testing – test the connection between the client, socket server, and database.
- System testing – test complete chat, group, presence, notification, and file-sharing flows.
- Acceptance testing – check the application against the acceptance criteria in the SRS.

Types:

- Functional testing – verify all functional requirements.
- Regression testing – repeat important tests after changes or fixes.
- Performance testing – check message latency and concurrent connections.
- Reliability testing – check reconnect and availability.
- Usability testing – check responsive UI and ease of sending the first message.
- Security testing – check WSS/TLS, authentication, authorization, input validation, and rate limiting.

Entry Criteria: A stable build is available, the test environment is ready, the socket server and database are working, and dummy test data is available.

Exit Criteria: All High-priority functional requirements are implemented and verified, there are no open Critical or Major defects (see Section 5.2), and the RTM shows that all mapped test cases have passed.

5.1 Security Validation

- Verify that passwords are strongly hashed and are not stored or logged in plaintext.
- Verify that TLS/WSS is used for client-server communication.

- Verify that a valid authentication token is required for the WebSocket handshake.
- Verify that users cannot access conversations or groups that they do not belong to.
- Test message-send rate limiting to reduce spam and socket flooding.
- Test input and file-upload validation using invalid and unsafe inputs.
- Test message and profile-field inputs for Cross-Site Scripting (XSS): submit script/HTML payloads (e.g., <script>, event-handler attributes) in chat messages, group names, and profile fields, and verify the UI renders them as plain text rather than executing them.
- Run an automated scan (e.g., OWASP ZAP baseline scan) against the REST endpoints as a supporting check alongside manual XSS and input-validation test cases.

5.2 Defect Severity & Priority Classification

Every logged defect is classified using the scale below so that "critical defect" in the Exit Criteria and in Section 14 metrics is unambiguous.

Severity	Definition	Example	Expected Fix Turnaround
Critical	Core function is broken, data is lost/corrupted, or a security control fails.	Messages fail to send; passwords stored in plaintext; auth bypass.	Before next test cycle / same day
Major	A required feature works incorrectly or a High-priority NFR/SR is not met, but there is a workaround.	Group messages delayed beyond NFR-001 target; read receipts not updating.	Within 1–2 days
Minor	A non-blocking functional issue with limited impact.	Typing indicator lingers after inactivity; unread count off by one.	Before next milestone
Cosmetic	UI/text/layout issue with no functional impact.	Misaligned button on mobile view; inconsistent spacing.	Best effort

6. Test Environment

Item	Details
Hardware	Standard laptops/desktops with network connectivity for client and server testing.
Software	Selected project stack: Node.js + WebSockets/Socket.IO or Python 3 + Flask-SocketIO; database such as MongoDB, PostgreSQL, or SQLite; modern web browsers.
Unit Test Framework	Jest or Mocha/Chai if the Node.js + Socket.IO stack is chosen; PyTest if the Python + Flask-SocketIO stack is chosen. Coverage is measured with Istanbul/nyc (Node.js) or pytest-cov (Python) to verify the ≥80% target in CHAT-NF-005.
Browser / Device Matrix	Desktop: Chrome and Firefox (latest stable) on Windows/macOS. Mobile: Chrome on Android (physical device or emulator) and Safari on iOS (physical device or simulator). Each matrix entry is used for the responsive-UI and first-message-within-3-taps checks in CHAT-NF-004.
Tools	Postman for REST API testing; WebSocket test client such as wscat/Socket.IO client; Artillery or JMeter for load testing; OWASP ZAP for baseline security scanning; browser DevTools for UI checks; GitHub Issues/Trello/Jira for defect tracking.

Item	Details
Test Data	Dummy user accounts, sample groups, text messages, file/image attachments, and admin/moderation test data.

7. Test Schedule

Milestones:

- Test case design: 27-Sep-2026
- Environment setup: 29-Sep-2026
- Test execution start: 30-Sep-2026
- Test execution end: 12-Oct-2026
- Review / Acceptance walkthrough: 14-Oct-2026 to 16-Oct-2026

Note: only a 1-day gap separates test execution end and the review walkthrough (see Section 10 for the associated risk and mitigation).

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual and automated)
- Test Scripts (API/socket automation)
- Test Data
- Test Execution Logs
- Defect Reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA / Testing Lead	Amoghaa R	Prepare the test plan, coordinate testing, and maintain test documentation.
Backend / Socket Lead	Vaibhav R	Design and execute server/socket tests and record backend defects.
Auth & Database Lead	P S Harshavardhan	Design and execute authentication and database tests and record defects.
Frontend / UI Lead	Pramath Ganesh Hegde	Design and execute UI and usability tests and record defects.
Course Coordinator		Review test results and approve the final test status.

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early builds and integrate modules step by step.

Risk	Mitigation
WebSocket connection problems	Use a stable test environment and repeat tests after the connection is stable.
Limited team size and academic timeline	Test High-priority requirements first and plan remaining tests carefully.
Test environment downtime	Keep a backup local environment or cloud VM.
Not enough clients for load testing	Use a load-testing tool that can create at least 100 WebSocket connections.
Only a 1-day buffer between test execution end (12-Oct) and the review walkthrough (14-Oct), leaving little time to fix defects found late	Triage and fix Critical/Major defects (Section 5.2) throughout execution rather than at the end; run rolling regression after each fix; if High-priority items are still failing by 11-Oct, extend execution and move the review walkthrough back rather than reviewing against a red RTM.

11. Assumptions & Dependencies

- A single technology stack is finalized before testing begins.
- Dummy accounts, groups, messages, and files are available before execution.
- The database is stable and available during testing.
- The client browser supports WebSockets.
- The test environment supports HTTPS/WSS and WebSocket connections.
- Attachment storage is available on local/server storage.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment is unavailable for more than 4 hours.
- The build is too unstable and blocks more than 30% of planned test cases.
- A blocking defect prevents meaningful execution of dependent tests.

Resume testing if:

- Blocking defects are resolved or an approved workaround is available.
- The test environment is stable.
- A stable build is available and passes basic smoke testing.

13. Test Case Management & Traceability

The Requirements Traceability Matrix (RTM) maps SRS requirements to test cases. The following are examples using the requirement and test case IDs defined for this project:

- CHAT-F-001 (Register user) → TC-Auth-01
- CHAT-F-010 (Send direct message) → TC-Msg-01
- CHAT-F-020 (Create group) → TC-Grp-01
- CHAT-F-040 (File/image sharing) → TC-File-01
- CHAT-NF-001 (Message latency) → TC-Perf-01
- CHAT-SR-001 (TLS/WSS) → TC-Sec-01

- CHAT-SR-003 (Authenticated WebSocket handshake) → TC-Sec-03

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed
- % passed / failed
- Defect density
- Defect aging
- Open defect count by severity (Critical / Major / Minor / Cosmetic, per Section 5.2)
- Requirement coverage
- p95 message delivery latency
- Number of concurrent WebSocket connections
- Server-side unit test coverage

Reports:

- Daily execution status
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA / Testing Lead	Amoghaa R	
Backend / Socket Lead	Vaibhav R	
Auth & Database Lead	P S Harshavardhan	
Frontend / UI Lead	Pramath Ganesh Hegde	
Course Coordinator		